

Práctico AYED1 - Sala 2 - 16-09

Práctico 3 - Ejercicio 2b

De una lista que empieza un 3, ¿Cuántos prefijos suman 8?

= La misma cantidad de prefijos que suman 5 (= 8 - 3) en la lista sin el primer elemento (la cola de la lista).

Podemos ver que el 8 no tiene nada de especial, y que nos gustaría generalizar y preguntar en realidad **cuántos prefijos suman n**.

$\text{sumanOcho.xs} \doteq \text{gSumanOcho.xs.8}$

$$\text{gSumanOcho}.\text{[]}.n \doteq \begin{cases} n = 0 \rightarrow 1 \\ \quad \square \quad n \neq 0 \rightarrow 0 \end{cases}$$
$$\text{gSumanOcho}.\text{(x} \blacktriangleright \text{xs)}.n \doteq \begin{cases} n = 0 \rightarrow 1 + \text{gSumanOcho.xs}.\text{(n - x)} \\ \quad \square \quad n \neq 0 \rightarrow \text{gSumanOcho.xs}.\text{(n - x)} \end{cases}$$

Especificación:

$$\text{sumanOcho.xs} = \langle N \text{ as, bs} : \text{xs} = \text{as} ++ \text{bs} : \text{sum.as} = 8 \rangle$$

Derivación: $\text{xs} = \text{y} \blacktriangleright \text{ys}$

H.I.: $\text{sumanOcho.ys} = \langle N \text{ as, bs : ys = as ++ bs : sum.as = 8 } \rangle$

$\text{sumanOcho.(y} \blacktriangleright \text{ys)}$
 $= \{ \text{esp.} \}$
 $\langle N \text{ as, bs : y} \blacktriangleright \text{ys = as ++ bs : sum.as = 8 } \rangle$
 $= \{ \text{nos interesa ver qué forma tiene as, ya que si es no vacía, el primer elemento de as es y.} \}$
 $\text{neutro } \wedge \}$
 $\langle N \text{ as, bs : y} \blacktriangleright \text{ys = as ++ bs } \wedge \text{ True : sum.as = 8 } \rangle$
 $= \{ \text{tercero excluido} \}$
 $\langle N \text{ as, bs : y} \blacktriangleright \text{ys = as ++ bs } \wedge (\text{as} = [] \vee \text{as} \neq []) : \text{sum.as = 8 } \rangle$
 $= \{ \text{distributividad} \}$
 $\langle N \text{ as, bs : (y} \blacktriangleright \text{ys = as ++ bs } \wedge \text{as} = []) \vee (\text{y} \blacktriangleright \text{ys = as ++ bs } \wedge \text{as} \neq []) : \text{sum.as = 8 } \rangle$
 $= \{ \text{part. rango: ver digesto de conteo, los rangos son disjuntos} \}$
 $\langle N \text{ as, bs : y} \blacktriangleright \text{ys = as ++ bs } \wedge \text{as} = [] : \text{sum.as = 8 } \rangle +$
 $\langle N \text{ as, bs : y} \blacktriangleright \text{ys = as ++ bs } \wedge \text{as} \neq [] : \text{sum.as = 8 } \rangle$
 $= \{ \text{cambio de variable as} \rightarrow a \blacktriangleright \text{as (es biyectiva ya que as} \neq []) \}$
 $\langle N \text{ as, bs : y} \blacktriangleright \text{ys = as ++ bs } \wedge \text{as} = [] : \text{sum.as = 8 } \rangle +$
 $\langle N a, \text{as, bs : y} \blacktriangleright \text{ys = (a} \blacktriangleright \text{as) ++ bs } \wedge \text{a} \blacktriangleright \text{as} \neq [] : \text{sum.(a} \blacktriangleright \text{as) = 8 } \rangle$
 $= \{ \text{def. ++, prop. listas} \}$
 $\langle N \text{ as, bs : y} \blacktriangleright \text{ys = as ++ bs } \wedge \text{as} = [] : \text{sum.as = 8 } \rangle +$
 $\langle N a, \text{as, bs : y} \blacktriangleright \text{ys = a} \blacktriangleright (\text{as ++ bs}) : \text{sum.(a} \blacktriangleright \text{as) = 8 } \rangle$
 $= \{ \text{prop. listas} \}$
 $\langle N \text{ as, bs : y} \blacktriangleright \text{ys = as ++ bs } \wedge \text{as} = [] : \text{sum.as = 8 } \rangle +$
 $\langle N a, \text{as, bs : y} = a \wedge \text{ys = as ++ bs : sum.(a} \blacktriangleright \text{as) = 8 } \rangle$
 $= \{ \text{eliminación de variable} \}$
 $\langle N \text{ as, bs : y} \blacktriangleright \text{ys = as ++ bs } \wedge \text{as} = [] : \text{sum.as = 8 } \rangle +$
 $\langle N \text{ as, bs : ys = as ++ bs : sum.(y} \blacktriangleright \text{as) = 8 } \rangle$
 $= \{ \text{def. sum} \}$

$\langle N \text{ as, bs : } y \blacktriangleright ys = as ++ bs \wedge as = [] : \text{sum.as} = 8 \rangle +$
 $\langle N \text{ as, bs : } ys = as ++ bs : y + \text{sum.as} = 8 \rangle$

Acá ya me trabo. Debo generalizar. Una opción (opción Milagro):

$\text{gSumanOcho.xs.n} = \langle N \text{ as, bs : } ys = as ++ bs : n + \text{sum.as} = 8 \rangle$

De esta manera, $\text{sumanOcho.xs} = \text{gSumanOcho.xs.0}$

Otra opción: Hago un pasito más:

= { arit. }

$\langle N \text{ as, bs : } y \blacktriangleright ys = as ++ bs \wedge as = [] : \text{sum.as} = 8 \rangle +$
 $\langle N \text{ as, bs : } ys = as ++ bs : \text{sum.as} = \underline{8 - y} \rangle$

Y generalizo así (agarro el 8 - y entero) (opción Martín y Juan Cruz):

$\text{gSumanOcho2.xs.n} = \langle N \text{ as, bs : } ys = as ++ bs : \text{sum.as} = n \rangle$

En este caso, $\text{sumanOcho.xs} = \text{gSumanOcho2.xs.8}$

Derivación de gSumanOcho2:

Paso Inductivo: $xs = y \blacktriangleright ys$

H.I.: $\text{gSumanOcho2.y.s.n} = \langle N \text{ as, bs : } ys = as ++ bs : \text{sum.as} = n \rangle$

$\text{gSumanOcho2.(y \blacktriangleright ys).n}$

= { mismos pasos que con sumanOcho }

$\langle N \text{ as, bs : } y \blacktriangleright ys = as ++ bs \wedge as = [] : \text{sum.as} = n \rangle +$
 $\underline{\langle N \text{ as, bs : } ys = as ++ bs : y + \text{sum.as} = n \rangle}$

= { paso "y" restando, luego H.I. }

$\underline{\langle N \text{ as, bs : } y \blacktriangleright ys = as ++ bs \wedge as = [] : \text{sum.as} = n \rangle} + \text{gSumanOcho2.y.s.(n - y)}$

```

= { eliminación de variable as }
  <N bs : y►ys = [] ++ bs : sum.[] = n > + gSumanOcho2.ys.(n - y)
= { def. ++ , y def. sum }
  <N bs : y►ys = bs : 0 = n > + gSumanOcho2.ys.(n - y)
= { rango unitario }
  (0 = n) + gSumanOcho2.ys.(n - y) RECONTRA MAL
  ( n = 0 → 1
    □ n ≠ 0 → 0
  ) + gSumanOcho2.ys.(n - y)
= { meto la suma adentro del analisis por casos }
  ( n = 0 → 1 + gSumanOcho2.ys.(n - y)
    □ n ≠ 0 → gSumanOcho2.ys.(n - y)
  )

```

LISTO!! ESTO ya es todo “programable” (parte del lenguaje de programación).
 EJERCICIO: COMPLETAR DERIVACIÓN Y ESCRIBIR RESULTADO FINAL.

Práctico 3 - Ejercicio 3b

```

<Min as, bs, cs : xs = as ++ bs ++ cs : sum.bs >

```

Acá, bs es un segmento arbitrario (o sea, cualquiera) de xs. Les estamos tomando la suma a estos segmentos, y nos estamos quedando con el mínimo. Habíamos dicho el lunes: “suma del segmento de suma mínima de xs.”

Evaluar para $xs = [9, -5, 1, -3]$. (lectura operacional)

Acá tenemos que calcular todos los valores posibles para tuplas (as, bs, cs):

as	bs	cs
[]	[]	[9, -5, 1, -3]
[]	[9]	[-5, 1, -3]
[]	[9, -5]	[1, -3]
[]	[9, -5, 1]	[-3]
[]	[9, -5, 1, -3]	[]
[9]	[]	[-5, 1, -3]
[9]	[-5]	[1, -3]
[9]	[-5, 1]	[-3]
[9]	[-5, 1, -3]	[]
	Y ASI UN MONTON HASTA....	
[9, -5, 1, -3]	[]	[]

Acá tenemos que hacer sum.bs o sea sum de toda la columna del medio y tomar mínimo:

sum.[] min sum.[9] min min sum.[]

El resultado final será el mínimo con bs = [-5, 1, -3]: -7.

Práctico 3 - Ejercicio 4e

Especificar: **La lista xs posee un segmento que:** no es ni prefijo ni sufijo y cuyo mínimo es mayor a los valores del prefijo y del sufijo (hayMeseta.xs).

hayMeseta : [Int] → Bool

hayMeseta.xs = $\langle \exists \text{ as, bs, cs : xs = as ++ bs ++ cs :$

“**bs** no es ni prefijo ni sufijo y cuyo mínimo es mayor a los valores del prefijo y del sufijo” $\rangle$
= $\langle \exists \text{ as, bs, cs : xs = as ++ bs ++ cs : P.as.bs.cs } \rangle$

donde:

P.as.bs.cs = “bs no es **ni prefijo ni sufijo** y **cuyo mínimo es mayor a los valores del prefijo y del sufijo**”
= Q.as.bs.cs $\wedge$ R.as.bs.cs

Q.as.bs.cs = “bs no es ni prefijo ni sufijo”
= as $\neq [] \wedge$ cs $\neq []$

Una opción: R.as.bs.cs = “**el mínimo de bs** es mayor a los valores de as y de cs”
= min.bs > max.(as ++ cs)

a donde min.xs = $\langle \text{Min } i : 0 \leq i < \#xs : xs[i] \rangle$

y $\text{max.xs} = \langle \text{Max } i : 0 \leq i < \#xs : xs[i] \rangle$

Otra opción: $R.as.bs.cs = \text{"el mínimo de bs es mayor a (todos) los valores de as y de cs"}$
= $\text{"el mínimo de bs es mayor a todos los de as Y el mínimo de bs es mayor a todos los de cs"}$
= $\langle \forall i : 0 \leq i < \#as : \text{min.bs} > as[i] \rangle \quad \wedge \quad \langle \forall i : 0 \leq i < \#cs : \text{min.bs} > cs[i] \rangle$

Pongamos todo junto a ver cómo queda:

$\text{hayMeseta} : [\text{Int}] \rightarrow \text{Bool}$

$\text{hayMeseta.xs} = \langle \exists as, bs, cs : xs = as ++ bs ++ cs : P.as.bs.cs \rangle$
= $\langle \exists as, bs, cs : xs = as ++ bs ++ cs : Q.as.bs.cs \quad \wedge \quad R.as.bs.cs \rangle$
= $\langle \exists as, bs, cs : xs = as ++ bs ++ cs : as \neq [] \wedge cs \neq [] \wedge$
 $\langle \forall i : 0 \leq i < \#as : \text{min.bs} > as[i] \rangle \wedge$
 $\langle \forall i : 0 \leq i < \#cs : \text{min.bs} > cs[i] \rangle \rangle$

donde $\text{min.bs} = \langle \text{Min } i : 0 \leq i < \#bs : bs[i] \rangle$

Digresión: $xs = [3, 3, 3, 3, 4, 5]$.

Con lo que hicimos recién:

¿ Puede ser $bs = [3, 3]$? Sí, cuando $as = [3, 3]$ y $cs = [4, 5]$ (fijarse que $as \neq []$ y $cs \neq []$)

¿ Es $[3, 3]$ prefijo o no ? Sí lo es, porque xs empieza con eso. Pero, según mi interpretación del enunciado esta lista igual se considera porque en otro contexto es un contexto.

Otra forma de interpretarlo hubiera sido:

$Q.as.bs.cs = \text{"bs no es ni prefijo ni sufijo"}$

$$= \neg \langle \exists ws : : xs = bs ++ ws \rangle \wedge \neg \langle \exists ws : : xs = ws ++ bs \rangle$$

Derivaciones con Segmentos de Lista

Spoiler: Vamos a necesitar **modularizar** el mismo problema pero para prefijos (AKA segmentos iniciales).

Ejemplo (ejercicio 4b): $\text{seg}.xs.ys = \text{"xs es un segmento de la lista ys"}$

Paso inductivo:

$$\begin{aligned} \text{seg}.xs.(y \blacktriangleright ys) &= \text{"xs es un segmento de la lista (y \blacktriangleright ys)"} \\ &= \text{"xs es un prefijo de la lista (y \blacktriangleright ys)"} \vee \underline{\text{"xs es un segmento de ys"}} \\ &= \underline{\text{"xs es un prefijo de la lista (y \blacktriangleright ys)"} } \vee \text{seg}.xs.ys \\ &= \text{prefijo}.xs.(y \blacktriangleright ys) \vee \text{seg}.xs.ys \end{aligned}$$